PROGRAMACIÓN II

Trabajo Práctico 3: Introducción a la

Programación Orientada a Objetos

Link Repositorio

Caso Práctico

Desarrollar en Java los siguientes ejercicios aplicando los conceptos de

programación orientada a objetos:

1. Registro de Estudiantes

a. Crear una clase Estudiante con los atributos: nombre, apellido, curso,

calificación.

Métodos requeridos: mostrarInfo(), subirCalificacion(puntos),

bajarCalificacion(puntos).

Tarea: Instanciar a un estudiante, mostrar su información, aumentar y disminuir

calificaciones.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package TPsModulo3;

/**
 *
 * @author ppasa
 */
public class Estudiante {

    private final String nombre;
    private final String apellido;
    private final String curso;
    private double calificacion;

    public Estudiante(String nombre, String apellido, String curso, double calificacion) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.curso = curso;
        this.calificacion = calificacion;
    }

    public void mostrarInfo() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Apellido: " + apellido);
        System.out.println("Curso: " + curso);
        System.out.println("Calificación: " + calificacion);
    }

    public void subirCalificacion(double puntos) {
        calificacion += puntos;
    }
     public void bajarCalificacion(double puntos) {
        calificacion -= puntos;
    }

    public static void main(String[] args) {
        Estudiante estudiante1 = new Estudiante("Juan", "Pérez", "Matemática", 7.5);
        System.out.println("Información inicial:");
        estudiante1.mostrarInfo();

        estudiante1.subirCalificacion(1.0);
        System.out.println("\nLuego de subir calificación:");
        estudiante1.mostrarInfo();

        estudiante1.bajarCalificacion(0.5);
        System.out.println("\nLuego de bajar calificación:");
        estudiante1.mostrarInfo();
    }
}
```

```
run:
Informaci►n inicial:
Nombre: Juan
Apellido: P►rez
Curso: Matem►tica
Calificaci►n: 7.5

Luego de subir calificaci►n:
Nombre: Juan
Apellido: P►rez
Curso: Matem►tica
Calificaci►n: 8.5

Luego de bajar calificaci►n:
Nombre: Juan
Apellido: P►rez
Curso: Matem►tica
Calificaci►n: 8.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

2. Registro de Mascotas

a. Crear una clase Mascota con los atributos: nombre, especie, edad.

Métodos requeridos: mostrarInfo(), cumplirAnios().

Tarea: Crear una mascota, mostrar su información, simular el paso del tiempo y verificar los cambios.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package TPsModulo3;

/**
 *
 * @author ppasa
 */
public class Mascota {
    private String nombre;
    private String especie;
    private int edad;

    public Mascota(String nombre, String especie, int edad) {
        this.nombre = nombre;
        this.especie = especie;
        this.edad = edad;
    }
public void mostrarInfo() {
    System.out.println("Nombre: " + nombre);
    System.out.println("Especie: " + especie);
    System.out.println("Edad: " + edad + " años");
}
public void cumplirAnios() {
    edad++;
    System.out.println("¡Feliz cumpleaños, " + nombre + "! Ahora tiene " + edad + " años.");
}
```

```java
public static void main(String[] args) {
    Mascota miMascota = new Mascota("Atila", "Perro", 10);

    System.out.println("Información de la mascota:");
        miMascota.mostrarInfo();

    System.out.println("\nSimulando el paso de un año...");
        miMascota.cumplirAnios();

    System.out.println("\nInformación actualizada de la mascota:");
        miMascota.mostrarInfo();
}

}
```

```
Output - Tps Programacio 2 (run)
run:
Informaci►n de la mascota:
Nombre: Atila
Especie: Perro
Edad: 10 a►os

Simulando el paso de un a►o...
►Feliz cumplea►os, Atila! Ahora tiene 11 a►os.

Informaci►n actualizada de la mascota:
Nombre: Atila
Especie: Perro
Edad: 11 a►os
BUILD SUCCESSFUL (total time: 0 seconds)
```

3. Encapsulamiento con la Clase Libro

a. Crear una clase Libro con atributos privados: titulo, autor, añoPublicacion.

Métodos requeridos: Getters para todos los atributos. Setter con validación para añoPublicacion.

Tarea: Crear un libro, intentar modificar el año con un valor inválido y luego con uno válido, mostrar la información final.

```java
package TPsModulo3;

/**
 *
 * @author ppasa
 */
public class Libros {

    public static void main(String[] args) {
        Libro librol = new Libro();


        librol.setTitulo("El Elegido");
        librol.setAutor("Andrew Gross");


        if (!librol.setAnioPublicacion(3000)) {
            System.out.println("Intento invalido: anioo 3000 no aceptado.");
        }

        if (librol.setAnioPublicacion(2016)) {
            System.out.println("Anio cambiado correctamente a 2016.");
        }


        System.out.println("Informacion final del libro:");
        System.out.println("Titulo: " + librol.getTitulo());
        System.out.println("Autor: " + librol.getAutor());
        System.out.println("Anio: " + librol.getAnioPublicacion());
    }
}
```

```java
class Libro {
    private String titulo;
    private String autor;
    private int anioPublicacion;

    public Libro() {
    }

    // Getters
    public String getTitulo() {
        return titulo;
    }

    public String getAutor() {
        return autor;
    }

    public int getAnioPublicacion() {
        return anioPublicacion;
    }

    // Setters
    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public void setAutor(String autor) {
        this.autor = autor;
    }

    public boolean setAnioPublicacion(int anioPublicacion) {
        int anioActual = java.time.Year.now().getValue();
        if (anioPublicacion > 0 && anioPublicacion <= anioActual) {
            this.anioPublicacion = anioPublicacion;
            return true;
        }
        return false; //
    }
}
```

```java
            this.anioPublicacion = anioPublicacion;
            return true;
        }
        return false; //
    }
}
```

```
utput - Tps Programacio 2 (run)
run:
Intento invalido: anioo 3000 no aceptado.
Anio cambiado correctamente a 2016.
Informacion final del libro:
Titulo: El Elegido
Autor: Andrew Gross
Anio: 2016
BUILD SUCCESSFUL (total time: 0 seconds)
```

4. Gestión de Gallinas en Granja Digital

a. Crear una clase Gallina con los atributos: idGallina, edad,

huevosPuestos.

Métodos requeridos: ponerHuevo(), envejecer(), mostrarEstado().

Tarea: Crear dos gallinas, simular sus acciones (envejecer y poner huevos), y mostrar su estado.

```java
package TPsModulo3;

/**
 *
 * @author ppasa
 */

public class GranjaDigital {
public static void main(String[] args) {

Gallina gallina1 = new Gallina(1, 2);
Gallina gallina2 = new Gallina(2, 1);

gallina1.ponerHuevo();
gallina1.ponerHuevo();
gallina1.envejecer();
gallina2.ponerHuevo();
gallina2.envejecer();
gallina2.ponerHuevo();

gallina1.mostrarEstado();
gallina2.mostrarEstado();
}
}
```

```java
class Gallina {

private int idGallina;
private int edad;
private int huevosPuestos;
// Constructor
public Gallina(int idGallina, int edad) {
this.idGallina = idGallina;
this.edad = edad;
this.huevosPuestos = 0;
}
// Método para simular que la gallina pone un huevo
public void ponerHuevo() {
huevosPuestos++;
System.out.println("La gallina " + idGallina + " ha puesto un huevo.");
}
// Método para simular que la gallina envejece
public void envejecer() {
edad++;
System.out.println("La gallina " + idGallina + " ha envejecido. Nueva edad: " + edad);
}
public void mostrarEstado() {
System.out.println("Gallina " + idGallina + ": Edad = " + edad + " años, Huevos puestos = " + huevosPuestos);
}
}
```

```
Output - Tps Programacio 2 (run)
run:
La gallina 1 ha puesto un huevo.
La gallina 1 ha puesto un huevo.
La gallina 1 ha envejecido. Nueva edad: 3
La gallina 2 ha puesto un huevo.
La gallina 2 ha envejecido. Nueva edad: 2
La gallina 2 ha puesto un huevo.
Gallina 1: Edad = 3 a►os, Huevos puestos = 2
Gallina 2: Edad = 2 a►os, Huevos puestos = 2
BUILD SUCCESSFUL (total time: 0 seconds)
```

5. Simulación de Nave Espacial

Crear una clase NaveEspacial con los atributos: nombre, combustible.

Métodos requeridos: despegar(), avanzar(distancia), recargarCombustible(cantidad), mostrarEstado().

Reglas: Validar que haya suficiente combustible antes de avanzar y evitar que

se supere el límite al recargar.

Tarea: Crear una nave con 50 unidades de combustible, intentar avanzar sin recargar, luego recargar y avanzar correctamente. Mostrar el estado al final.

```java
package TPsModulo3;

/**
 *
 * @author ppasa
 */
public class NaveEspacial {
    private String nombre;
    private int combustible;
    private final int LIMITE_COMBUSTIBLE = 100;
    public NaveEspacial(String nombre, int combustible) {
    this.nombre = nombre;
    if (combustible > LIMITE_COMBUSTIBLE) {
    this.combustible = LIMITE_COMBUSTIBLE;
} else {
    this.combustible = combustible;
}
}
    public void despegar() {
        if (combustible >= 10) {
        combustible -= 10;
        System.out.println(nombre + " ha despegado. Combustible restante: " + combustible);
}   else {
    System.out.println("No hay suficiente combustible para despegar.");
}
}
    public void avanzar(int distancia) {
    int consumo = distancia * 2;
    if (combustible >= consumo) {
    combustible -= consumo;
    System.out.println(nombre + " avanzó " + distancia + " unidades. Combustible restante: " + combustible);
}   else {
    System.out.println("Combustible insuficiente para avanzar " + distancia + " unidades.");
}
}
    public void recargarCombustible(int cantidad) {
    if (combustible + cantidad > LIMITE_COMBUSTIBLE) {
    System.out.println("No se puede recargar esa cantidad. Se superaría el límite de combustible.");
} else {
    combustible += cantidad;
    System.out.println("Combustible recargado. Combustible actual: " + combustible);
}
}
```

```java
    public void mostrarEstado() {
    System.out.println("Nave: " + nombre + " | Combustible: " + combustible);
}
    public static void main(String[] args) {
    NaveEspacial nave = new NaveEspacial("Atila", 50);
    nave.despegar(); // Intenta despegar
    nave.avanzar(30); // Intenta avanzar sin recargar (debería fallar si no hay suficiente)
    nave.recargarCombustible(40); // Recarga combustible
    nave.avanzar(20); // Ahora debería poder avanzar
    nave.mostrarEstado(); // Muestra el estado final
}
}
```

```
Output - Tps Programacio 2 (run)
run:
Atila ha despegado. Combustible restante: 40
Combustible insuficiente para avanzar 30 unidades.
Combustible recargado. Combustible actual: 80
Atila avanz▯ 20 unidades. Combustible restante: 40
Nave: Atila | Combustible: 40
BUILD SUCCESSFUL (total time: 0 seconds)
```

[Link Repositorio]